Computability, Logic, and Randomness

Lecture Notes

Draft from October 27, 2026.
Check the website for the latest version.
Comments welcome!

Levin Hornischer
Levin.Hornischer@lmu.de
https://levinhornischer.github.io/CoLoR/

Contents

Preface 1

1. Introduction 3
1.1. Exercises . . . 8

I. Computability 9

2. Formalizing computation: Turing machines 10
2.1. Turing machines . . . 10
2.2. Church–Turing thesis . . . 14
2.3. Enumerating Turing machines . . . 14
2.4. Why partial functions? . . . 14
2.5. Universal Turing machines . . . 14
2.6. Exercises . . . 14

3. Incomputability 15
3.1. The Halting problem . . . 15
3.2. Rice's theorem . . . 15
3.3. Tiling problems . . . 15
3.4. Exercises . . . 15

4. Turing degrees: Ordering problems by difficulty 16
4.1. Computably enumerable sets . . . 16
4.2. Reducibility . . . 16
4.3. Turing degrees . . . 16
4.4. Exercises . . . 16

II. Logic 17

III. Randomness 18

Bibliography 19

Index 20

Preface

This is the reader for the course "Computability, Logic, and Algorithmic Randomness" given during the winter semester 2026/7 at *LMU Munich* as part of the *Master in Logic and Philosophy of Science*. These lecture notes are updated as the course progresses. A website with all the course material is found at

`https://levinhornischer.github.io/CoLoR/`.

Comments I'm happy about any comments: spotting typos, finding mistakes, pointing out confusing parts, or simply questions triggered by the material. Just email me at `Levin.Hornischer@lmu.de`.

Course description and objectives This course is an introduction to the theory around three interconnected concepts: computability, logic, and randomness. We start with the first concept: Computability theory describes which problems can be solved by an algorithm. This is made precise via so-called Turing machines, the theoretical underpinning of modern computers. Moving to the second concept, we see that not only can computability theory be used to answer questions in logic (e.g., prove Gödel's incompleteness theorem), but logic can be also used to informatively describe computational problems. Finally, time permitting, we look at the third concept: namely, randomness and its interplay with computability. We ask whether access to randomness helps computing power, or if purely random information is not useful information.

Prerequisites A prerequisite is an introductory course in first-order logic (soundness and completeness theorems, basic model theory) and some familiarity with mathematics (basic probability theory and some order theory), including the ability of writing mathematical proofs.

Layout These notes are informal and partially still under construction. For example, there are margin notes to convey more casual comments that you'd rather find in a lecture but usually not in a book. Todo notes indicate, well, that something needs to be done. References are found at the end. Exercises are at the end of each chapter.

This is a margin note.

This is a todo note

Study material Computability theory is a classic topic, so there are many textbooks available. Here are some:

There also is a discussion on math stackexchange on textbooks for computability theory.

- H. Rogers Jr. (1967). *Theory of Recursive Functions and Effective Computability*. New York: McGraw–Hill Book Company
- N. Cutland (1980). *Computability: An Inroduction to recursive function theory*. Cambridge: Cambridge University Press
- P. Odifreddi (1990). *Classical Recursion Theory: The Theory of Functions and Setsof Natural Numbers*. Studies in Logic and the Foundations ofMathematics, vol. 125. Amsterdam: North-Holland Publishing Company
- S. B. Cooper (2004). *Computability Theory*. Boca Raton,FL: Chapman & Hall/CRC
- G. S. Boolos et al. (2007). *Computability and Logic*. 5th ed. Cambridge: Cambridge University Press
- R. I. Soare (2016). *Turing Computability: Theory and Applications*. Berlin, Heidelberg: Springer. DOI: https://doi.org/10.1007/978-3-642-31933-4
- B. Robič (2020). *The Foundations of Computability Theory*. Berlin Heidelberg: Springer. DOI: 10.1007/978-3-662-62421-0
- G. Tourlakis (2022). *Computability*. Cham: Springer. DOI: 10.1007/978-3-030-83202-5

They also stress the tight connection with logic. For more, you can also consult Peter Smith's helpful "Beginning Mathematical Logic: A Study Guide", section 6 on "Arithmetic, computability, and incompleteness". There also are many lecture notes; in particular, I recommend those of Andrew Marks.

For a category-theoretic take on computability: Yanofsky (2022).

For algorithmic randomness, the classic reference is

- R. G. Downey and D. R. Hirschfeldt (2010). *Algorithmic Randomness and Complexity*. New York: Springer

and especially for the theme on interactions between randomness and computability, see

- A. Nies (2009). *Computability and Randomness*. Oxford: Oxford University Press

Notation Throughout, 'iff' abbreviates 'if and only if'.

1. Introduction

In this short introductory chapter, we motivate and overview the topics that we will cover in this course.

Computability theory studies the question: Which problems can be solved by a computer? Synonyms for 'computer' here are 'algorithm' or 'effective procedure'. To give a rigorous answer, the vague terms first have to be made precise (i.e., be *explicated* or *rationally reconstructed*, as philosophers would say). This will be done as follows.

1. A *problem* will be formally analyzed as a partial function $f : \mathbb{N} \rightharpoonup \mathbb{N}$: an input n encodes an instance of the problem and $f(n)$ encodes its solution, if it exists.
2. A *computer* will be formally analyzed as a Turing machine.
3. A computer *solving* a problem will be formally analyzed as the Turing machine computing a function which is identical to the function describing the problem.

Why partial functions and not, as usual, total functions? Intuitively, not every problem instance might have a solution. But there also formal reasons, which we discuss in section 2.4.

Historically, what gave rise to these questions was Hilbert's program. This program was Hilbert's response to the foundational crisis that mathematics faced at the beginning of the 20th century. To put mathematics on solid foundations, the suggestion was to fix a set of axioms and inference rules such that all—and only all—theorems of mathematics are derived from these axioms with those rules, ideally in an automated way. In the above terminology, we first enumerate all the mathematical statements and then the problem is: Given the input n, output 0 or 1 depending on whether the n-th statement is a theorem.

This is a very rough sketch of Hilbert's program; for more accuracy and details, see Zach (2023).

Gödel's incompleteness theorem famously shattered Hilbert's program, and together with the work of Turing, Church, and others computability theory was started. We now sketch the main insights, before doing this formally and in detail in the next chapters.

You might reflect on similarities and dissimilarities to the turmoil in mathematics caused by AI, now about a century later.

Problems and solutions As mentioned, a problem is formalized as a partial function $f : \mathbb{N} \rightharpoonup \mathbb{N}$. This may seem quite restricted. Indeed, this does not include problems like 'What is the meaning of life?'. But

it includes much more problems than it might initially seem. It includes all 'discrete' problems, i.e., those for which each problem instance and its solution can be described symbolically—like in Hilbert's case above. This is because each symbolic problem instance and each possible solution can uniquely be encoded by a natural number, so the problem becomes a partial function $f : \mathbb{N} \rightharpoonup \mathbb{N}$ which, on input n, is defined iff n encodes a problem instance, in which case $f(n)$ encodes the solution. But what would it mean to solve such a problem with a computer?

Turing machines In a remarkable feat of mathematical philosophy, Alan Turing explicated in 1936 the intuitive notion of (symbolic) computation with an abstract machine, now known as the *Turing machine*. It processes symbols, that are written on a tape, according to its program, which is simple a table of instructions. At any given time step, the machine is in some internal state and positioned at some cell of the tape. Based on its internal state and the symbol written in that cell, the program says (a) in which internal state the machine should go into next, (b) which symbol the machine should write into the present cell instead, and (c) whether it should move to the left or the right cell next. The machine has one designated internal state known as the halting state: if it enters that, it stops its computation.

The Turing machine then computes the following function: It takes as input a string of symbols (consisting only of symbols that also the machine uses). They are written on the otherwise blank tape. Then the machine start processing according to its program. If it enters its halting state, then the string of symbols that is written on the tape at that moment is the output of the function. If the machine never enters a halting state and 'loops forever', then no output is ever given.

Church–Turing thesis Why should that be 'the correct' way of formalizing computation? Other models of computation have been developed: e.g., Church's lambda calculus or Gödel's recursive functions. Remarkably, though, they all turned out to be equivalent: a function is computable in one model if and only if it is computable in the other model. This has lead to the conviction that the formal concept of Turing computability really coincides with the intuitive concept of (symbolic) computabilility. This is known as the *Church–Turing thesis*: that a function is Turing computable if and only if it is computable in the intuitive sense. Note that this is a *philosophical* and not a mathematical claim. It is a claim about an informal

philosophical concept: namely, that of computability. It cannot be proven by formal mathematical means.

Methods to establish (non-) computability Having established a formalization of our initial question—namely, which problems are solvable by a computer?—, the next question is how we can establish whether or not a problem is computable.

The expected functions indeed turn out to be computable. For example, the addition function that maps a code for a pair of numbers (n, m) to the number $n + m$. Similarly for multiplication, exponentiation, ordering, etc. But Turing already was also asking: are there functions that are not computable?[1]

In other words, are there things that no computer can ever solve? Turing considered the so-called *halting problem*: Given a Turing machine e and an input n, decide if the Turing machine on this input halts or not. A Turing machine that solves this problem is a 'meta-program' that checks whether an inputted programs ever gets stuck in a loop. So it would be very desirable to have such a meta-program, but Turing showed that this cannot exist.

Theorem 1.1 (Turing)**.** *There is no Turing machine that solves the halting problem. In other words, the* h *function that maps a (code of a) Turing machine* e *and an input* n *to*

$$h(e, n) := \begin{cases} 1 & \textit{Turing machine } e \textit{ halts on input } n \\ 0 & \textit{otherwise} \end{cases}$$

is not computable.

Proof. Toward a contradiction, assume h is computable. Then we can define the Turing machine e that, on input n, computes $h(n, n)$ and if this is 0 it outputs 0 and otherwise loops forever. Now, we plug e into h. Since h always outputs 0 or 1, there are two cases:

- Case 1: $h(e, e) = 0$. Then, by definition, the Turing machine e with input e outputs 0, i.e., halts, so $h(e, e) = 1$.
- Case 2: $h(e, e) = 1$. Then, by definition, the Turing machine e with input e runs forever, i.e., does not halt, so $h(e, e) = 0$.

[1] Turing machines are finite objects, there are only countably many Turing machines, so there are also only countably many computable functions. On the other hand, there are uncountably many function $\mathbb{N} \to \mathbb{N}$ (by Cantor's diagonalization theorem). So there must be *some* uncomputable functions. But explicitly providing one is more difficult.

Thus, we indeed get a contradiction. □

Knowing the non-computability of the halting problem is a stepping stone for other non-computability results. A common method to prove a given problem to be non-computable is a *reduction argument*: assume there is a Turing machine solving the given problem and the use this to build a Turing machine that solves the halting problem. Since the latter is impossible, the former is, too.

This is how the historically important *Entscheidungsproblem* was answered in the negative: there is no algorithm to decide whether a given formula in first-order logic is logically valid (i.e., true in all interpretations). If there was such an algorithm, we could solve the halting problem: given Turing machine e and input n, there is a clever way to build a formula φ of first-order logic such that φ is valid iff e halts on input n, so if we can decide validity we can also decide halting.

In fact, also Gödel's first incompleteness theorem can be proven by a reduction argument. (Gödel's original proof was different: Turing machines did not exist yet at that time.)

The formulation that we state here uses the concept of a computable enumeration. A set of string is *computably enumerable* if there is a Turing machine that, as it keeps processing, writes on the tape strings separated by a blank cell such that all written strings are in the set and each string of the set eventually is written on the tape. This is weaker than the set of string being *decidable*: for that we require a Turing machine that take a string as input and outputs 'yes' or 'no' depending on whether or not the string is in the set. In the computably enumerable case, we only get confirmation if the string is in the set. If the string is not in the set, then, as we follow the enumeration, we never know if the string actually is in the set but has not yet appeared or actually is not in the set and because of that did not appear. On the other hand, in the decidable case, we also get confirmation if the string is not in the set.

This version is discussed, e.g., in this online lecture.

Theorem 1.2 (Gödel's first incompleteness theorem)**.** *There is no computably enumerable list of axioms that prove all and only the true statements of elementary mathematics (including, say, basic arithmetic, finite combinatorics, etc.).*

Just arithmetic is enough, because other 'elementary mathematics' can be encoded with natural numbers, so questions there reduce to questions about arithmetic.

Proof. For a reduction argument, assume there is such a list of axioms. By systematically applying all the inference rules, we can computably enumerate all and only the theorems of the theory (not just the axioms). But then we can solve the halting problem: Given a Turing machine e and an input n, consider the statement 'Turing machine e halts on input n'.

This is a statement of elementary mathematics. So either this statement or its negation shows up on the enumeration of all theorems (one of the two has to). So we start the enumeration and wait until one of the two shows up, and depending on which, we say the Turing machine halts or does not halt. This hence would be a computable procedure to solve the halting problem, which is impossible. □

This shattered Hilbert's dream of writing down, in a systematic—and hence computably enumerable—way all the axioms of mathematics, so that every mathematical truth can be immutably established by deriving it from the axioms.

Turing degrees By now, we have seen computable problems as well as non-computable problems, and the method of reducing one problem to another. This invites the general question: What structure does the collection of all problems have, ordered by reduction? This is the structure of the Turing degrees. It measure the 'degree of unsovability' of problems. The higher up a problem is in this order, the harder it is to solve. Hence much research went into understanding this order, with stunning results.

For example, every countable partial order embeds into the order of Turing degrees, so they are, in a sense, as complex as can be. Also, the first-order theory of the Turing degrees is very complicated: it is as hard as true second-order arithmetic. Finally, the answer to Post's problem shows that there are degrees strictly between the computable problems and the halting problem.

Randomness So far, we have seen interactions between computability and logic. But what about the third concept in the course's title? Roughly, randomness is the polar opposite of computability: Consider the outcome of an infinite sequence of coin tosses $f : \mathbb{N} \to \{0, 1\}$, i.e., $f(n) = 0$ (resp., $= 1$) means that the n-th toss was heads (resp., tails). This sequence is random, so we should not be able to predict with some algorithm what comes next, i.e., the function f should not be computable. The field of algorithmic randomness studies when a sequence is random. Thus, by understanding randomness, we also gain insights about its opposite, computability.

More specifically, we study the relationship between computability and randomness by asking whether randomness is a resource to computability. If the Turing machine may use the outcome of a sequence of coin tosses in its computation, can it, for some nonzero probability of coin tosses, correctly solve the halting problem? The answer is no, which is a theorem

by de Leeuw, Moore, Shannon, and Shapiro and, independently, Sacks. Intuitively, then, while a single random sequence contains a lot of information (since it is random, we cannot compress it into any sequence with fewer information), they do not, on average, contain any 'useful' information with which we can compute more than what we could compute with deterministic Turing machines.

This is for computability in general. Randomness might still help a computing agent with finite memory: see Icard (2019).

However, we may push further and ask if this not only holds 'on average' but also for individual sequences: Is it true that random sequences contain a lot of information but no useful information? Here the answer is no for arguably the most popular formalization of randomness known as Martin-Löf randomness. One particular sequence, known as Chaitin's Ω, is Martin-Löf random but it can compute the halting problem. (And the Kučera–Gács theorem show that in fact every sequence can be computed from some Martin-Löf random sequence.) But this ceases to be the case for the stronger notions of randomness (like difference randomness or weak 2-randomness). So the interplay between computability and randomness is rather delicate!

1.1. Exercises

Exercise 1.a. We said that computability theory consider 'discrete' problems $f : \mathbb{N} \rightharpoonup \mathbb{N}$ where problem instances and their solutions can be described symbolically. But what about computing with real numbers, i.e., computing functions $f : \mathbb{R} \rightharpoonup \mathbb{R}$? Can they be captured on the symbolic point of view (e.g., as floating point real numbers)? Or do we need a different theory?

For more on this, see, e.g., Pour-El and Richards 1989.

Exercise 1.b. Here we only considered abstract mathematical models of Turing machines. We do not consider what it means to implement or realize a Turing machine in a physical system, like my laptop. Think about what such a 'realization' relation between the theoretical model and a concrete system involves.

For more on this, see, e.g., Gandy (1980) and Sieg (2002).

Part I.

Computability

2. Formalizing computation: Turing machines

In this chapter, we introduce Turing machines to formalize the intuitive idea of computation.

to be updated

2.1. Turing machines

We now formalize the informal description of a Turing machine discussed in chapter 1. We fix the symbol $\llcorner\!\lrcorner$ and call it the *blank symbol*.

Definition 2.1. A *Turing machine* M is a tuple $(S, Q, q_{\text{start}}, q_{\text{halt}}, \delta)$ where

- S is a finite set with $\llcorner\!\lrcorner, 1 \in S$. We call S the *alphabet* of M, and we call the elements of S *symbols* of M. We call the elements of $S \setminus \{\llcorner\!\lrcorner\}$ the *non-blank symbols* of M.
- Q is a finite set with $q_{\text{start}}, q_{\text{halt}} \in Q$. We call the elements of Q the (*internal*) *states* of M, we call q_{start} the *starting state*, and we call q_{halt} the *halting state*.
- $\delta : S \times Q \rightharpoonup S \times Q \times \{L, R\}$ is a partial function. We call δ the *transition map* or *program* of M.

A *configuration* (or *overall state*) of M is a how the machine can 'look like' at any given moment: precisely, it is a triple (σ, q, n) where

- $\sigma : \mathbb{Z} \to S$ is a function, with $\sigma(k) \in S$ being the symbol written at position $k \in \mathbb{Z}$ of the tape,
- $q \in Q$ is the internal state of the machine, and
- $n \in \mathbb{Z}$ is the position on the tape where the machine is located.

A *run* of M is a finite or infinite sequence

$$(\sigma_0, q_0, n_0), (\sigma_1, q_1, n_1), (\sigma_2, q_2, n_2), \ldots$$

of configurations of M such that

- The machine starts in the starting state at position 0, i.e., $q_0 = q_{\text{start}}$ and $n_0 = 0$, while σ_0 describes what is written on the tape at the start.

- For all i, if q_i is the halting state q_{halt}, then (σ_i, q_i, n_i) is the last entry of the sequence, and the runs ends in the halting state. If q_i is not the halting state, there are two cases. If the program $\delta(\sigma_i(n_i), q_i))$ is not defined for the currently read symbol and state, then the machine stalls but does not halt, and the run ends with (σ_i, q_i, n_i) but not in a halting state. If the program $\delta(\sigma_i(n_i), q_i)) = (s, q, X)$ is defined, then the next configuration $(\sigma_{i+1}, q_{i+1}, n_{i+1})$ must be such that $q_{i+1} = q$ and

$$n_{i+1} = \begin{cases} n_i + 1 & \text{if } X = R \\ n_i - 1 & \text{if } X = L \end{cases} \qquad \sigma_{i+1}(k) = \begin{cases} s & \text{if } k = n_i \\ \sigma_i(k) & \text{otherwise.} \end{cases}$$

There are many variants of this definition. For example, we could refrain from requiring the symbol 1, we could require also a symbol 0, we could have one fixed starting state and initial state (like the blank symbol, rather than making it part of the signature of a Turing machine), we could even allow several tapes, or instead of L and R we could allow also N for staying put on the present position of the tape. However, this would not change what the Turing machines are able to do: because one variant can simulate the other variant. For example, staying put (N) amounts to the same as first moving right (R) and then left (L). One might need a few more steps, but ultimately can achieve the same. Formally, they compute the same function, in the following sense.

See exercise 2.a.

Definition 2.2. A function $f : \mathbb{N}^k \to \mathbb{N}$ is (*Turing-*) *computable* if, for all $(x_1, \ldots, x_k) \in \mathbb{N}^k$, the following holds:

- If $f(x_1, \ldots, x_k) = y$ is defined, then, when the machine starts with the configuration $(\sigma_0, q_{\text{start}}, 0)$ where σ_0 looks as follows

$$\ldots \sqcup\sqcup \underbrace{\sqcup}_{\text{position } 0} \overbrace{1\ldots1}^{x_1 \text{ many}} \sqcup \overbrace{1\ldots1}^{x_2 \text{ many}} \sqcup \ldots \sqcup \overbrace{1\ldots1}^{x_k \text{ many}} \sqcup\sqcup \ldots,$$

  (with the dots indicating blank cells), then the run of the machine will end in a configuration $(\sigma, q_{\text{halt}}, n)$ where σ is of the form

$$\ldots \sqcup \overbrace{1\ldots1}^{y \text{ many}} \underbrace{\sqcup}_{\text{position } n} \ldots$$

  where all unspecified cells (those indicated by the dots) can contain

whatever symbol.

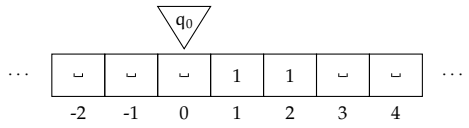
- If $f(x_1, \ldots, x_k)$ is not defined, then, when the machine starts with the configuration $(\sigma_0, q_{\text{start}}, 0)$ as above, the run of the machine will not end in a halting state (either stalls or runs forever).

Example 2.3. Consider the following Turing machine that computes the function $f : \mathbb{N} \to \mathbb{N}$ given by $f(x) = x + 1$, i.e., the successor function.

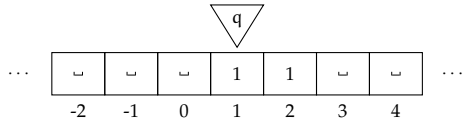
Let $S = \{\text{␣}, 1\}$, let $Q = \{q_0, q_1, q\}$ with $q_{\text{start}} = q_0$ and $q_{\text{halt}} = q_1$, and let δ be given as follows:

S	Q	S	Q	{L, R}
␣	q_0	␣	q	R
1	q	1	q	R
␣	q	1	q_1	R

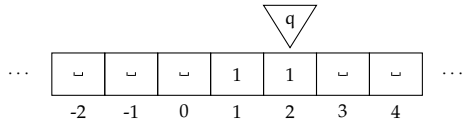
Let's see how the machine processes the input $x = 2$, for which it should produce the output $f(x) = 3$. The starting configuration looks as follows:



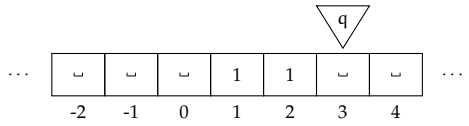
Hence the first row of the instruction table applies, so we write a blank, go into state q and move the head right. So the next configuration is



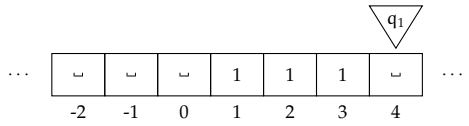
Now the second row of the instruction table applies, so we write a 1, go into (or rather stay in) state q, and move the head right. So the next configuration is



Now still the second row of the instruction table applies, so we write a 1, go into q, and move the head right. So the next configuration is

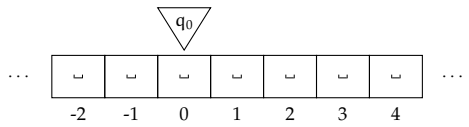


Now the third row of the instruction table is applicable, so we write a 1, go into state q_1, and move the head right. So the next configuration is

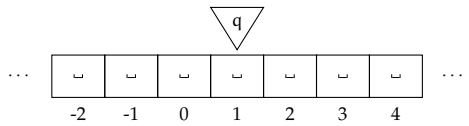


Now we have reached the halting state, so the run stops here in the halting state. We can read off the output. It is indeed of the form required by definition 2.2: To the left of the current position of the head, there is a sequences of 1's, surrounded by two blank symbols, with the head being positioned on the right end of the sequence. The sequence consists of three 1's, representing the intended output $f(x) = 3$.

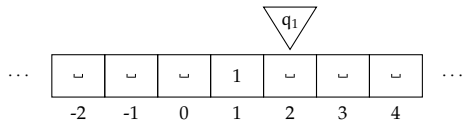
Other inputs work similarly. However, it is good to consider the 'edge case' $x = 0$. There, the starting configuration looks as follows:



Hence again the first row of the instruction table applies, so we write a blank, go into state q and move the head right. So the next configuration is



Now the third row of the instruction table is applicable, so we write a 1, go into state q_1, and move the head right. So the next configuration is



Now we have reached the halting state and the run stops. The tape again is in the required form to read off the input: a sequence of 1's surrounded by blanks, with the head being positioned on the right end. The sequence of 1's has length one, representing the intended output $f(x) = 1$. ⌟

Exercise 2.b asks you to do more of such examples.

2.2. Church–Turing thesis

2.3. Enumerating Turing machines

2.4. Why partial functions?

2.5. Universal Turing machines

2.6. Exercises

Exercise 2.a. Think about how the variants of the definition of a Turing machine (listed after definition 2.1) are equivalent in the sense that one variant can be simulated in the other.

Exercise 2.b. Show that the following functions are Turing-computable by explicitly defining Turing machines a proving that they compute the respective function:

1. $f : \mathbb{N} \to \mathbb{N}, f(x) := 0$.
2. $f : \mathbb{N}^2 \to \mathbb{N}, f(x, y) := x + y$.
3. $f : \mathbb{N} \to \mathbb{N}, f(x) := 2y$.

It is helpful to first describe in plain English how the Turing machine program should work and then write down the actual table δ.

3. Incomputability

3.1. The Halting problem

3.2. Rice's theorem

3.3. Tiling problems

3.4. Exercises

4. Turing degrees: Ordering problems by difficulty

4.1. Computably enumerable sets

4.2. Reducibility

4.3. Turing degrees

4.4. Exercises

Part II.

Logic

Part III.

Randomness

Bibliography

Boolos, G. S., J. P. Burgess, and R. C. Jeffrey (2007). *Computability and Logic*. 5th ed. Cambridge: Cambridge University Press (cit. on p. 2).

Cooper, S. B. (2004). *Computability Theory*. Boca Raton,FL: Chapman & Hall/CRC (cit. on p. 2).

Cutland, N. (1980). *Computability: An Inroduction to recursive function theory*. Cambridge: Cambridge University Press (cit. on p. 2).

Downey, R. G. and D. R. Hirschfeldt (2010). *Algorithmic Randomness and Complexity*. New York: Springer (cit. on p. 2).

Gandy, R. (1980). "Church's Thesis and Principles for Mechanisms." In: *The Kleene Symposium*. Ed. by J. Barwise, H. J. Keisler, and K. Kunen. Amsterdam: North-Holland Publishing Company, pp. 123–148 (cit. on p. 8).

Icard, T. (11/2019). "Why Be Random?" In: *Mind* 130.517, pp. 111–139. DOI: 10.1093/mind/fzz065 (cit. on p. 8).

Nies, A. (2009). *Computability and Randomness*. Oxford: Oxford University Press (cit. on p. 2).

Odifreddi, P. (1990). *Classical Recursion Theory: The Theory of Functions and Setsof Natural Numbers*. Studies in Logic and the Foundations ofMathematics, vol. 125. Amsterdam: North-Holland Publishing Company (cit. on p. 2).

Pour-El, M. B. and I. J. Richards (1989). *Computability in Analysis and Physics*. Perspectives in Mathematical Logic. Berlin, Heidelberg: Springer (cit. on p. 8).

Robič, B. (2020). *The Foundations of Computability Theory*. Berlin Heidelberg: Springer. DOI: 10.1007/978-3-662-62421-0 (cit. on p. 2).

Rogers Jr., H. (1967). *Theory of Recursive Functions and Effective Computability*. New York: McGraw–Hill Book Company (cit. on p. 2).

Sieg, W. (2002). "Calculations by man and machine: conceptual analysis." In: *Reflections on the foundations of mathematics: Essays in honor of Solomon Feferman*. Ed. by W. Sieg, R. Sommer, and C. Talcott. Vol. 15. Lecture Notes in Logic. Natick, Massachusetts: Association for Symbolic Logic, pp. 390–409 (cit. on p. 8).

Soare, R. I. (2016). *Turing Computability: Theory and Applications*. Berlin, Heidelberg: Springer. DOI: https://doi.org/10.1007/978-3-642-31933-4 (cit. on p. 2).

Tourlakis, G. (2022). *Computability*. Cham: Springer. DOI: 10.1007/978-3-030-83202-5 (cit. on p. 2).

Yanofsky, N. S. (2022). *Theoretical Computer Science for the Working Category Theorist*. Cambridge Elements. Cambridge: Cambridge University Press (cit. on p. 2).

Zach, R. (2023). "Hilbert's Program." In: *The Stanford Encyclopedia of Philosophy*. Ed. by E. N. Zalta and U. Nodelman. Winter 2023. Metaphysics Research Lab, Stanford University (cit. on p. 3).